

ATK-MB022 模块使用说明

OLED 模块

使用说明

正点原子

广州市星翼电子科技有限公司

修订历史

版本	日期	原因
V1.0	2024/11/01	第一次发布

目 录

1, 硬件连接.....	1
2, 实验功能.....	2
2.1 OLED 模块测试实验	2
2.1.1 功能说明.....	2
2.1.2 源码解读.....	2
2.1.3 实验现象.....	10
3, 其他.....	11

1，硬件连接

这里以正点原子 M48Z-M3 最小系统板 STM32F103 版为例，给大家介绍一下模块和板卡的连接方法。其它板卡与模块的硬件连接方法，请大家在“**ATK-MB022 OLED 模块3，程序源码\相应板卡例程文件夹\readme.txt**”路径下查看。

OLED 模块可通过杜邦线与正点原子 M48Z-M3 最小系统板 STM32F103 版进行连接，具体的连接关系，如下表所示：

模块对应开发板	连接关系			
	VCC	GND	SDA	SCL
M48Z-M3 最小系统板 STM32F103 版	3.3V/5V	GND	PA3	PA2

表 1.1.1 OLED 模块与 M48Z-M3 最小系统板 STM32F103 版连接关系

2，实验功能

2.1 OLED 模块测试实验

2.1.1 功能说明

在本实验中，开发板主控芯片通过模拟 IIC 与 OLED 模块通信，下载程序成功后，按下复位键，OLED 模块会自动循环显示 ASCII 字符及其对应的码值，并通过串口打印相关提示信息。

开发板的 LED0 闪烁，提示程序运行。

2.1.2 源码解读

打开本实验的工程文件夹，能够在./Drivers/BSP 目录下看到 ATK_OLED 文件夹，和 IIC 文件夹，其中 ATK_OLED 文件夹中就包含了 OLED 模块的驱动文件和字库文件，IIC 文件夹中就包含了软件模拟 IIC 的驱动文件如下图所示：

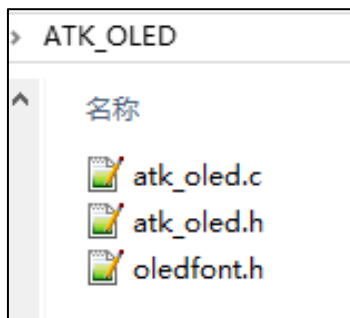


图 2.1.2.1 OLED 模块驱动代码

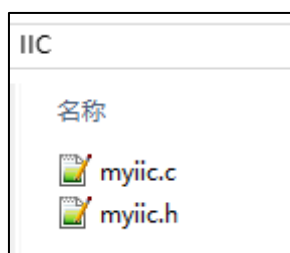


图 2.1.2.2 IIC 驱动代码

2.1.2.1 OLED 模块驱动

OLED 驱动源码包括：atk_oled.c、atk_oled.h 和 oledfont.h，oledfont.h 头文件存放的是 ASCII 字符集，atk_oled.h 存放的是引脚接口宏定义和函数声明等，atk_oled.c 则是驱动代码。

首先看 oledfont.h 头文件的 ASCII 字符集内容：

```
/* 常用 ASCII 表
* 偏移量 32
* ASCII 字符集： !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNPOQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
* PC2LCD2002 取模方式设置： 阴码+逐列式+顺向+C51 格式
* 总共： 3 个字符集（12*12、 16*16 和 24*24） ， 用户可以自行新增其他分辨率的字符集。
```

```
* 每个字符所占用的字节数为: (size/8+((size%8)?1:0))*(size/2),
其中 size:是字库生成时的点阵大小(12/16/24...)
*/
/* 12*12 ASCII 字符集点阵 */
const unsigned char oled_asc2_1206[95][12]={ ...这里省略字符集库... };
/* 16*16 ASCII 字符集点阵 */
const unsigned char oled_asc2_1608[95][16]={ ...这里省略字符集库... };
/* 24*24 ASCII 字符集点阵 */
const unsigned char oled_asc2_2412[95][36]={ ...这里省略字符集库... };
下面将简要介绍 atk_oled.c 和 oledfont.h 中几个重要的 API 函数。
```

该头文件中包含三个大小不同的 ASCII 字符集点阵，其中包括：12*12 ASCII 字符集点阵、16*16 ASCII 字符集点阵、24*24 ASCII 字符集点阵。每个字符集点阵都包含 95 个常用的 ASCII 字符集，从空格符开始，分别为： !"#\$%&'()*+,-0123456789;:<=>?@ABCD EFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~。

上面的 ASCII 字符集，我们可以使用正点原子团队开发的一个款很好的字符提取软件来制作获取。字符提取软件为：ATK_XFONT，该软件可以提供各种字符，包括汉字（字体和大小都可以自己设置）阵提取，且取模方式可以设置好几种，常用的取模方式，该软件都支持。该软件还支持图形模式，也就是用户可以自己定义图片的大小，然后画图，根据所画的图形再生成点阵数据，这功能在制作图标或图片的时候很有用。

该软件的界面如下图所示：

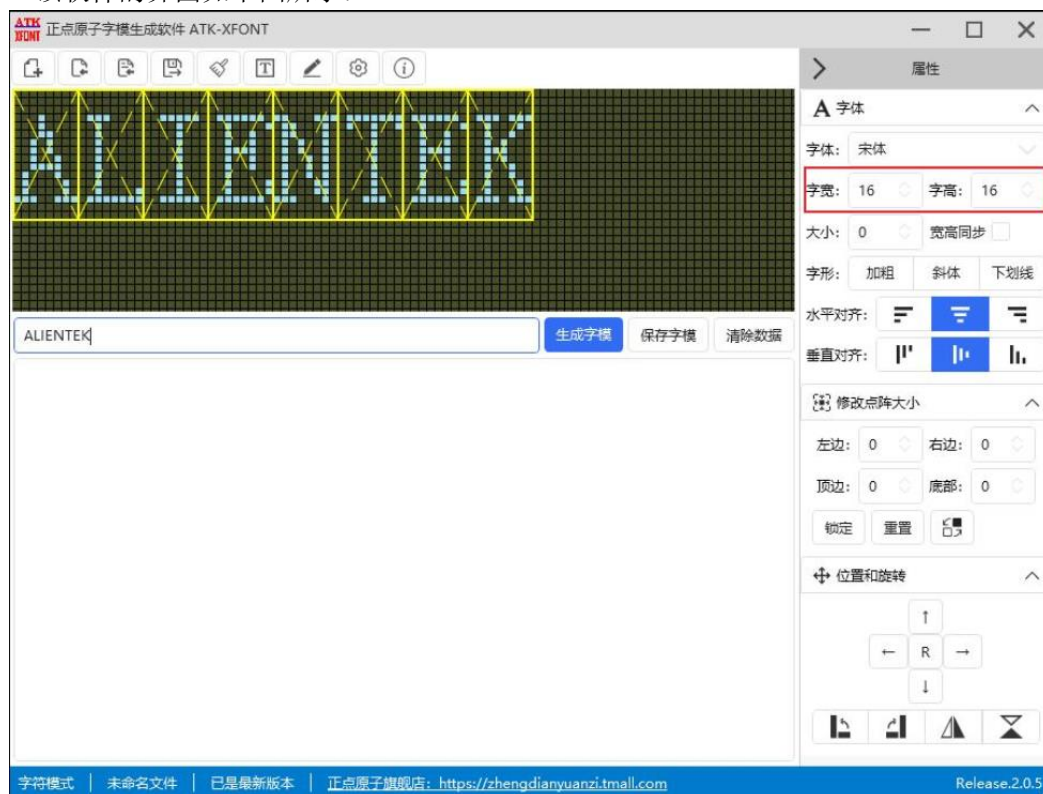


图 2.1.2.1.1 ATK_XFONT 软件界面

然后我们选择设置，在设置里面设置取模方式如下图所示：

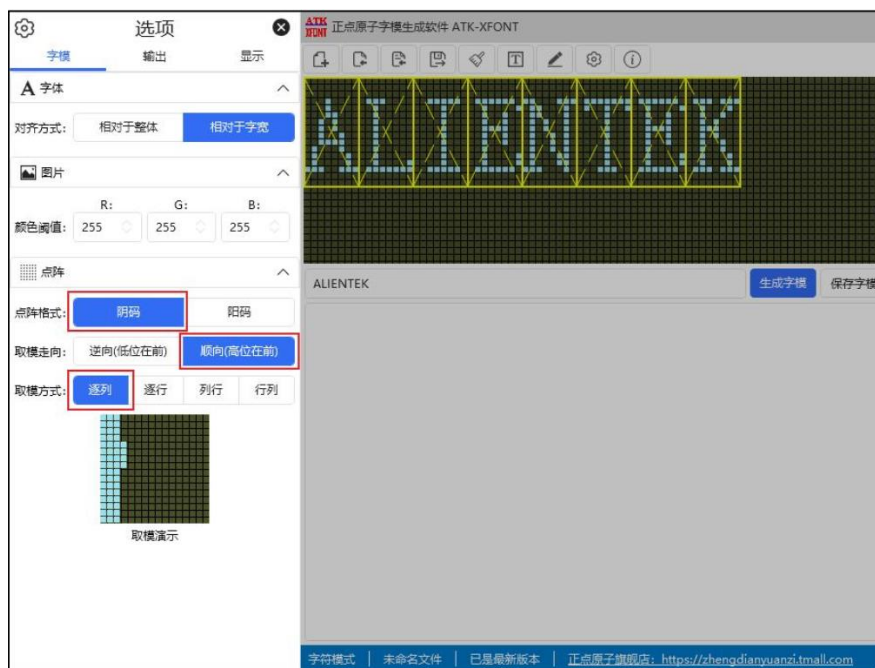


图 2.1.2.1.2 设置取模方式

上图设置的取模方式，在右上角的取模说明里面有，即从第一列开始向下每取 8 个点作为一个字节，如果最后不足 8 个点就补满 8 位。取模顺序是从高到低，即第一个点作为最高位。如*-----取为 10000000。其实就是按如图 2.1.2.1.3 所示的这种方式：

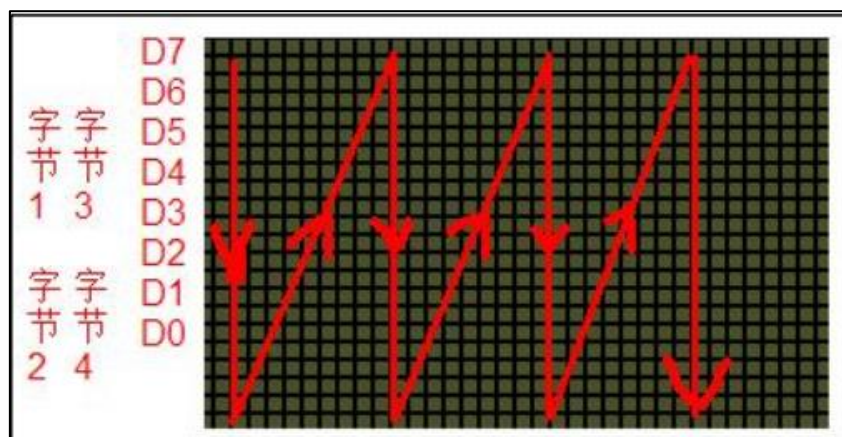


图 2.1.2.1.3 取模方式图解

从上到下，从左到右，高位在前。我们按这样的取模方式，然后把 ASCII 字符集按 12*6 大小、16*8 和 24*12 大小取模出来（对应汉字大小为 12*12、16*16 和 24*24，字符的只有汉字的一半大！），每个 12*6 的字符占用 12 个字节，每个 16*8 的字符占用 16 个字节，每个 24*12 的字符占用 36 个字节。

atk_oled.c 文件中的代码帮助我们在 OLED 屏幕上显示自定义字符集。以下是 atk_oled.c 中几个重要的 API 函数的简要介绍。

1. 函数 atk_oled_init()

该函数用于初始化 OLED 模块，具体的代码，如下所示：

```
/**
 * @brief      初始化 OLED(SSD1306)
 * @param      无
 * @retval     无
```

```

*/
void atk_oled_init(void)
{
    iic_init();

    atk_oled_wr_byte(0xAE, OLED_CMD); /* 关闭显示 */
    atk_oled_wr_byte(0xD5, OLED_CMD); /* 设置时钟分频因子,震荡频率 */
    atk_oled_wr_byte(80, OLED_CMD); /* [3:0],分频因子;[7:4],震荡频率 */
    atk_oled_wr_byte(0xA8, OLED_CMD); /* 设置驱动路数 */
    atk_oled_wr_byte(0X3F, OLED_CMD); /* 默认 0X3F(1/64) */
    atk_oled_wr_byte(0xD3, OLED_CMD); /* 设置显示偏移 */
    atk_oled_wr_byte(0X00, OLED_CMD); /* 默认为 0 */

    atk_oled_wr_byte(0x40, OLED_CMD); /* 设置显示开始行 [5:0],行数. */

    atk_oled_wr_byte(0x8D, OLED_CMD); /* 电荷泵设置 */
    atk_oled_wr_byte(0x14, OLED_CMD); /* bit2, 开启/关闭 */
    atk_oled_wr_byte(0x20, OLED_CMD); /* 设置内存地址模式 */
    /* [1:0], 00, 列地址模式; 01, 行地址模式; 10, 页地址模式; 默认 10; */
    atk_oled_wr_byte(0x02, OLED_CMD);
    atk_oled_wr_byte(0xA1, OLED_CMD); /* 段重定义设置, bit0: 0, 0->0; 1, 0->127; */
    /* 设置 COM 扫描方向; bit3: 0, 普通模式; 1, 重定义模式 COM[N-1]->COM0; N: 驱动路数 */
    atk_oled_wr_byte(0xC8, OLED_CMD);
    atk_oled_wr_byte(0xDA, OLED_CMD); /* 设置 COM 硬件引脚配置 */
    atk_oled_wr_byte(0x12, OLED_CMD); /* [5:4] 配置 */

    atk_oled_wr_byte(0x81, OLED_CMD); /* 对比度设置 */
    atk_oled_wr_byte(0xEF, OLED_CMD); /* 1~255; 默认 0X7F (亮度设置, 越大越亮) */
    atk_oled_wr_byte(0xD9, OLED_CMD); /* 设置预充电周期 */
    atk_oled_wr_byte(0xf1, OLED_CMD); /* [3:0], PHASE 1; [7:4], PHASE 2; */
    atk_oled_wr_byte(0xDB, OLED_CMD); /* 设置 VCOMH 电压倍率 */
    /* [6:4] 000, 0.65*vcc; 001, 0.77*vcc; 011, 0.83*vcc; */
    atk_oled_wr_byte(0x30, OLED_CMD);
    /* 全局显示开启; bit0: 1, 开启; 0, 关闭; (白屏/黑屏) */
    atk_oled_wr_byte(0xA4, OLED_CMD);
    atk_oled_wr_byte(0xA6, OLED_CMD); /* 设置显示方式; 1, 反相显示; 0, 正常显示 */
    atk_oled_wr_byte(0xAF, OLED_CMD); /* 开启显示 */
    atk_oled_clear();
}

```

从上述代码可以看出, 首先通过初始化 I2C 接口, 确保开发板能够通过 I2C 通信与 OLED 模块进行交互。接下来是一些 OLED 模块的初始化指令序列, 我们按照厂家提供的参考资料逐步进行配置即可。

2. 函数 atk_oled_refresh_gram()

该函数实现了将显存 (g_atk_oled_gram) 中的内容更新到 OLED 屏幕上显示, 具体

代码，如下所示：

```
static uint8_t g_atk_oled_gram[128][8];

/**
 * @brief      更新显存到 OLED
 * @param      无
 * @retval     无
 */
void atk_oled_refresh_gram(void)
{
    uint8_t i, n;

    for (i = 0; i < 8; i++)
    {
        atk_oled_wr_byte (0xb0 + i, OLED_CMD); /* 设置页地址 (0~7) */
        atk_oled_wr_byte (0x00, OLED_CMD);      /* 设置显示位置-列低地址 */
        atk_oled_wr_byte (0x10, OLED_CMD);      /* 设置显示位置-列高地址 */

        for (n = 0; n < 128; n++)
        {
            atk_oled_wr_byte(g_atk_oled_gram[n][i], OLED_DATA);
        }
    }
}
```

从上面的代码中可以看出，首先在函数外部定义一个静态二维数组 `g_atk_oled_gram[128][8]`，大小为 128 列 × 8 页（屏幕的像素按页和列划分），用于存储 OLED 的显示内容。在函数 `atk_oled_refresh_gram()` 中通过设置页地址和列地址，将显存中的内容逐页、逐列写入 OLED 显示屏，最终完成整块屏幕的刷新。。

3. 函数 `atk_oled_wr_byte()`

该函数实现了通过 I2C 协议向 OLED 屏幕写入一个字节的数据或命令，具体代码，如下所示：

```
/**
 * @brief      向 OLED 写入一个字节
 * @param      data: 要写入的数据/命令
 * @param      cmd: 数据/命令标志 0, 表示命令; 1, 表示数据;
 * @retval     无
 */
static void atk_oled_wr_byte(uint8_t data, uint8_t cmd)
{
    iic_start();
    iic_send_byte(OLED_I2C_ADDR);
    iic_ack();
    cmd? iic_send_byte(0x40) : iic_send_byte(0x00);
}
```



```
iic_ack();
iic_send_byte(data);
iic_ack();
iic_stop();
}
```

从上述代码可以看出，首先通过 I2C 协议向 OLED 模块写入一个字节的的数据或命令。根据 `cmd` 参数，函数可以选择发送命令 (`cmd == 0`) 或数据 (`cmd == 1`)。完整的流程包括：发送设备地址、发送标志位（数据或命令）、发送实际字节，最后结束通信。

4. 函数 `atk_oled_draw_point()`

该函数实现了在 OLED 屏幕任意位置上绘制单个点的功能，具体代码，如下所示：

```
/**
 * @brief      OLED 画点
 * @param      x   : 0~127
 * @param      y   : 0~63
 * @param      dot: 1 填充 0, 清空
 * @retval     无
 */
void atk_oled_draw_point(uint8_t x, uint8_t y, uint8_t dot)
{
    uint8_t pos, bx, temp = 0;

    if (x > 127 || y > 63) return; /* 超出范围了. */

    pos = y / 8; /* 计算 GRAM 里面的 y 坐标所在的字节，每个字节可以存储 8 个行坐标 */

    bx = y % 8; /* 取余数，方便计算 y 在对应字节里面的位置，及行 (y) 位置 */
    temp = 1 << bx; /* 高位表示高行号，得到 y 对应的 bit 位置，将该 bit 先置 1 */

    if (dot) /* 画实心点 */
    {
        g_atk_oled_gram[x][pos] |= temp;
    }
    else /* 画空点，即不显示 */
    {
        g_atk_oled_gram[x][pos] &= ~temp;
    }
}
```

从上述代码可以看出，该函数用于在 OLED 屏幕的指定位置绘制或清除一个像素点。通过输入横坐标 `x` (0-127) 和纵坐标 `y` (0-63)，该函数计算点在 OLED 显存中的位置，然后根据 `dot` 参数决定是否显示该点 (`dot = 1`) 或清除该点 (`dot = 0`)。它通过操作显存中的位数据实现像素级控制，灵活调整 OLED 屏幕上图像的显示。

5. 函数 `atk_oled_show_char()`

前面我们知道取模方式是：从上到下，从左到右，高位在前。下面根据取模的方式来编写显示字符 `atk_oled_show_char` 函数，其定义如下：

```
/**
 * @brief      在指定位置显示一个字符,包括部分字符
 * @param      x    : 0~127
 * @param      y    : 0~63
 * @param      size: 选择字体 12/16/24
 * @param      mode: 0,反白显示;1,正常显示
 * @retval     无
 */
void atk_oled_show_char(uint8_t x, uint8_t y, uint8_t chr, uint8_t size, uint8_t
mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;
    uint8_t *pfont = 0;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    uint8_t csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
    chr = chr - ' '; /* 得到偏移后的值,因为字库是从空格开始存储的,第一个字符是空格 */

    if (size == 12)          /* 调用 1206 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1206[chr];
    }
    else if (size == 16)     /* 调用 1608 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1608[chr];
    }
    else if (size == 24)     /* 调用 2412 字体 */
    {
        pfont = (uint8_t *)oled_asc2_2412[chr];
    }
    else                     /* 没有的字库 */
    {
        return;
    }

    for (t = 0; t < csize; t++)
    {
        temp = pfont[t];

        for (t1 = 0; t1 < 8; t1++)
        {
            if (temp & 0x80) atk_oled_draw_point(x, y, mode);
            else atk_oled_draw_point(x, y, !mode);
        }
    }
}
```

```
temp <=<= 1;
y++;

if ((y - y0) == size)
{
    y = y0;
    x++;
    break;
}
}
```

该函数为字符以及字符串显示的核心部分，函数中 `chr = chr - ' '`；这句是要得到在字符点阵数据里面的实际地址，因为我们的取模是从空格键开始的，例如 `oled_asc2_1206[0][0]`，代表的是空格符开始的点阵码。在接下来的代码，我们也是按照从上到下（先 `y++`），从左到右（再 `x++`）的取模方式来编写的，先得到最高位，然后判断是写 1 还是 0，然后进行画点；接着读第二位，如此循环，直到一个字符的点阵全部取完为止。这其中涉及到列地址和行地址的自增，根据取模方式来理解，就不难了。由于函数较多，关于 `atk_oled.c` 文件中的其他函数，请大家自行理解。

2.1.2.2 IIC 驱动

在图 2.1.2.2 中，`myiic.c` 和 `myiic.h` 是开发板与 OLED 模块通讯而使用的模拟 IIC 驱动文件，关于模拟 IIC 的驱动介绍，请查看正点原子各个开发板对应的开发指南中模拟 IIC 对应的章节。

2.1.2.3 实验测试代码

实验的测试代码为文件 `demo.c`，在工程目录下的 `User` 子目录中。测试代码的入口函数为 `demo_run()`，具体的代码，如下所示：

```
/**
 * @brief      例程演示入口函数
 * @param      无
 * @retval     无
 */
void demo_run(void)
{
    uint8_t t = 0;

    atk_oled_init(); /* 初始化 OLED */

    atk_oled_show_string(0, 0, "ALIENTEK", 24);
    atk_oled_show_string(0, 24, "0.96' OLED TEST", 16);
    atk_oled_show_string(0, 52, "ASCII:", 12);
    atk_oled_show_string(64, 52, "CODE:", 12);
    atk_oled_refresh_gram(); /* 更新显示到 OLED */

    t = ' ';
```

```
while (1)
{
    atk_oled_show_char(36, 52, t, 12, 1);    /* 显示 ASCII 字符 */
    atk_oled_show_num(94, 52, t, 3, 12);     /* 显示 ASCII 字符的码值 */
    atk_oled_refresh_gram();                 /* 更新显示到 OLED */
    t++;

    if (t > '~')
    {
        t = ' ';
    }

    delay_ms(500);
    LED0_TOGGLE();                          /* LED0 闪烁 */
}
}
```

demo_run 函数的流程大致：在 OLED 上显示一些实验信息字符，然后开始从空格键开始不停的循环显示 ASCII 字符集，并显示该字符的 ASCII 值。最后 LED0 闪烁提示程序正在运行。

2.1.3 实验现象

将 OLED 模块按照第一节“硬件连接”中介绍的连接方式与开发板连接，并将实验代码编译烧录至开发板中，本实验会在 OLED 屏幕显示实验现象，如下图所示：



图 2.1.3.1 OLED 模块显示内容

从上图可以看到 OLED 显示了三种尺寸的字符：24*12 (ALIENTEK)、16*8 (0.96' OLED TEST) 和 12*6 (剩下的内容)。说明我们的实验是成功的，实现了三种不同尺寸 ASCII 字符的显示，在最后一行不停的显示 ASCII 字符以及其码值。

3，其他

1、购买地址：

天猫：<https://zhengdianyuanzi.tmall.com>

淘宝：<https://openedv.taobao.com>

2、资料下载

模块资料下载地址：<http://www.openedv.com/docs/index.html>

3、技术支持

公司网址：www.alientek.com

技术论坛：<http://www.openedv.com/forum.php>

在线教学：www.yuanzige.com

B 站视频：<https://space.bilibili.com/394620890>

传真：020-36773971

电话：020-38271790

